

Face Recognition in Python and OpenCV

Student: Tudor-Andrei, Negrea

Coordinator: Radu Stefan, Ricman

Table of Contents

INTRODUCTION	3
THEORETICAL PART	5
PRACTICAL PART	9
CONCLUSIONS	16
REFERENCES	17
ANNEX: PROJECT PYTHON CODE.....	18

INTRODUCTION

1.1 Project Purpose and Objectives

In the contemporary landscape of artificial intelligence, computer vision stands as one of the most transformative domains, shifting human-computer interaction (HCI) from rigid command-line inputs to intuitive, cognitive-aware systems. Among these advancements, automated facial analytics and real-time expression evaluation have emerged as vital components in modern software ecosystems. This project implements an optimized, cross-platform **Real-Time Facial Expression and Emotion Recognition System**.

The primary objective of this system is to bridge deep convolutional neural networks (CNNs) with a highly stable, interactive desktop interface. By automating the extraction of micro-expressions, this application solves a critical problem in software engineering: making advanced deep learning insights accessible, responsive, and clear for practical use without requiring expensive cloud infrastructure or specialized hardware.

The application serves a wide variety of modern operational domains:

- **User Experience (UX) Research:** Tracking software testers' implicit emotional frustrations or satisfaction drops in real time.
- **Mental Health Telehealth Monitoring:** Assisting healthcare practitioners with objective, longitudinal data regarding patient affect states during video consultations.
- **Interactive Entertainment and Gaming:** Modifying in-game environmental assets or difficulty spikes based on the real-time emotional state of the player.
- **Intelligent Security and Smart Environments:** Enhancing situational awareness frameworks by detecting atypical or highly stressed emotional profiles in public access areas

1.2 Structural Workflows

To accommodate different deployment environments, the application is engineered with two distinct runtime workflows that handle pixel parsing and neural network execution differently:

1. **Static Image Archive Analysis:** This sub-system allows users to browse local file directories through an integrated native operating system file explorer. It accepts standard compressed graphic formats (.jpg, .jpeg, .png, .bmp, .webp). Once an asset is selected, a dedicated image processing pipeline intercepts the file stream, handles resizing constraints, extracts structural data, and runs an offline deep neural inference loop. The results are then dynamically presented within an isolated visual modal window.
2. **Dynamic Live Video Processing:** This sub-system establishes a high-frequency connection to the host hardware's primary video capture interface (e.g., integrated or external USB webcam). The incoming raw data stream is parsed frame-by-frame. A dual-layer algorithmic cascade maps facial regions and passes the resulting pixel arrays to an emotion evaluation engine, superimposing graphics and classification labels directly onto the live feed.

Static Processing Mode

User Choice → OS File Dialog → Disk Buffer → Pillow Scale → DeepFace Backend → Modal Pop-up

Dynamic Live Mode

Webcam Loop → Frame Grab → Grayscale Convert → Haar Cascade → Modulo Throttling → Softmax Output → OpenCV Overlay

1.3 Core Technological Frameworks

The system architecture relies on an optimized mix of user interface libraries, matrix manipulation toolkits, and abstract deep learning wrappers:

- **Python Language Environment:** Chosen due to its superior ecosystem for scientific computing, cross-platform compatibility, and excellent memory allocation features when wrapping low-level C++ computer vision binaries.

- **CustomTkinter:** A sophisticated UI wrapper built over Python's native Tkinter library. It provides a modern, hardware-accelerated dark theme widget toolkit. It uses responsive layout grids and explicit window object trees to eliminate UI stuttering during background calculations.
- **OpenCV (Open Source Computer Vision Library):** A highly optimized open-source library that handles real-time hardware data streams, color space manipulation (such as BGR to Grayscale mapping), and the mathematical rendering of bounding vectors and text strings directly onto pixel buffers.
- **DeepFace:** A powerful hybrid deep learning framework for Python that wraps state-of-the-art neural networks. It provides pre-trained weights for advanced models like VGG-Face, Google FaceNet, and OpenFace, allowing the application to extract deep facial metrics without needing expensive training phases.
- **Pillow (PIL):** An advanced image processing library utilized to manipulate raw data streams, manage image buffers, and scale assets down to strict aspect ratios without causing memory leaks.

THEORETICAL PART

2.1. Code Development and Dependency Inventory

Developing a real-time computer vision system requires a clean separation between the user interface thread and the analytical backend tasks. If the deep learning inference loop ran sequentially within the visual interface loop, the desktop layout would freeze every time a face was analyzed. To prevent this, the code uses an event-driven model that maps specific UI actions to background worker tasks.

The application relies on five key imports to establish its workspace:

```
import customtkinter as ctk
from tkinter import messagebox, filedialog
import cv2
import os
from deepface import DeepFace
from PIL import Image
```

By isolating these imports, the system preserves a modular structure where ctk and PIL govern visual presentation, while cv2 and DeepFace drive the core algorithmic processing.

Deep Learning Backend and Feature Extraction

The core analytical engine uses the DeepFace library to evaluate emotion states. When a pixel matrix is submitted for analysis, it passes through a multi-layered Convolutional Neural Network (CNN) optimized for facial attribute classification. This process relies on three sequential phases:

Input Matrix → Spatial Face Isolation → Normalized Sub-Region → CNN
Feature Map Layers → Softmax Classifier

1. **Spatial Region Extraction:** The input matrix is evaluated to find the boundaries of the human face. If no face is explicitly found, the system uses an `enforce_detection=False` safeguard parameter. This prevents a complete application crash by allowing the system to fall back to analyzing the overall image context.
2. **Dimensional Layer Operations:** The isolated face region is resized and fed into stacked convolutional layer sets. These layers apply mathematical filters to extract structural facial markers, such as the arc of an eyebrow, wrinkles around the eyes, or the curvature of the mouth.
3. **Softmax Vector Probability Generation:** The final layer of the network outputs a dense array of seven raw scores (logits), corresponding to standard human emotional states: *Anger*, *Disgust*, *Fear*, *Happiness*, *Sadness*, *Surprise*, and *Neutrality*. These values are normalized into a clean probability distribution using a Softmax activation function.

Mathematical Formulations

Spatial Coordinate Mapping for Face Bounding Boxes

When a face is identified within a frame, its location is mapped using a standard two-dimensional pixel coordinate plane. The object detection engine returns a raw array containing four bounding parameters: the origin coordinates (x, y) , the bounding width w , and the bounding height h .

The boundaries of the tracking rectangle are mathematically defined by two coordinates:

$$\begin{aligned}\text{Top-Left Corner} &= (x, y) \\ \text{Bottom-Right Corner} &= (x + w, y + h)\end{aligned}$$

Where:

- $x \in [0, \text{Max Horizontal Resolution}]$: Represents the pixel column coordinate starting from the left edge of the frame matrix.
- $y \in [0, \text{Max Vertical Resolution}]$: Represents the pixel row coordinate starting from the top edge of the frame matrix.
- w : Represents the calculated width of the detected facial region in pixels.
- h : Represents the calculated height of the detected facial region in pixels.

These bounding parameters are mapped directly into the OpenCV canvas renderer using the following implementation statement:

```
cv2.rectangle(frame, (x, y), (x + w, y + h), box_color, 2)
```

Emotion Probability Vector Distribution (Softmax Regression)

To turn raw neural network scores into clear emotion categories, the system uses Softmax regression. The probability P that a given facial region belongs to a specific emotional class c out of a total set of K classes is calculated using the following formula:

$$P(y = c \mid z) = \frac{e^{z_c}}{\sum_{j=1}^K e^{z_j}}$$

Where:

- z is the vector of raw, unnormalized model scores (logits) produced by the network's final layer.
- z_c is the specific score assigned to the target emotion class c .
- K represents the total number of distinct emotional categories ($K = 7$ for standard DeepFace emotion models).

The denominator $\sum_{j=1}^K e^{z_j}$ sums the exponential values of all scores, normalizing the output so that the total sum of all probabilities equals exactly 1.0 (or 100%). To output a single classification string to the UI, the system filters this probability array using an Argmax optimization operation:

$$\text{Dominant Emotion} = \arg \max_c P(y = c | z)$$

This extracts the string key matching the highest probability index, which is then capitalized and displayed on screen via `res['dominant_emotion'].upper()`.

Algorithmic Processing Throttling (Modulo Congruence)

Running a deep convolutional neural network on every single incoming video frame can easily overload local hardware, leading to a choppy video feed and delayed interface responses. To fix this, the script uses a mathematical throttling equation built on modular arithmetic.

Deep learning analysis is forced to run only when the frame counter matches a specific mathematical condition:

$$\text{Analyze Frame} \Leftrightarrow \text{counter} \equiv 0(\text{mod}20)$$

Which is evaluated in Python as:

```
if counter % 20 == 0 and len(faces) > 0:
```

Where:

- counter represents a strictly increasing integer variable that increments by 1 with each loop cycle.
- $(\text{mod}20)$ isolates the remainder after dividing the current frame count by a fixed period of 20.

This constraint splits the application's processing into two speeds. Lightweight spatial face tracking runs smoothly at full speed (e.g., 30 frames per second), while the heavy emotion recognition neural network runs at a lower, hardware-friendly frequency (1.5 calculations per second).

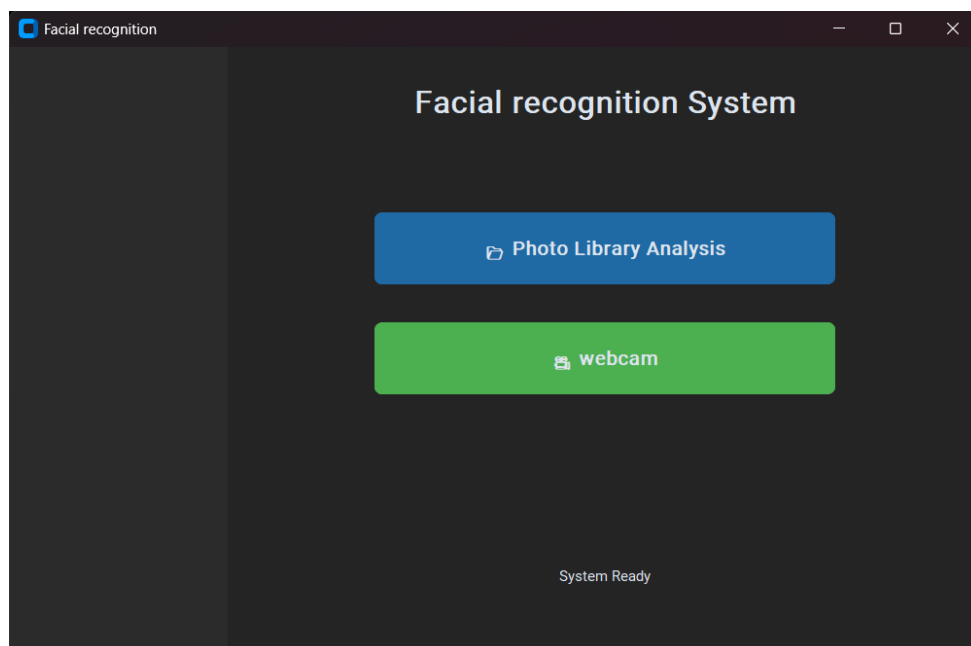
PRACTICAL PART

3.1 Functional Implementation and Architecture

The practical implementation of this application is structured as an object-oriented Python class called FaceApp, which inherits from customtkinter.CTk. This approach encapsulates the application state, layout widgets, and hardware handlers within a single execution context.

The software layout is broken down into clean functional components:

- **Window Configurations:** The constructor defines window properties, locking the default size to 800×500 pixels and forcing a clean grid layout configuration.
- **The Modern Sidebar:** A dedicated visual frame (CTkFrame) pinned to the left edge. It is reserved for future updates, such as adding database logs or multi-camera navigation links.
- **The Control Panel Canvas:** A transparent main frame that holds the primary header text, a large photo analysis launch button, a green webcam activation button, and a real-time status tracker at the bottom.



3.2 Graphical User Interface Component Mapping

Quantitative tracking and structural mapping are essential for assessing how a computer vision application manages its underlying frameworks and user interactions. The following tables detail the internal configuration of the software, breaking it down into structural dependencies and operational lifecycles.

Library / Module Name	Framework Purpose	Key Classes & Methods Utilized	Functional Responsibility
customtkinter (ctk)	Graphical User Interface	CTk(), CTkButton(), CTkToplevel(), grid()	Manages the main dark-mode application window, button styling, and layout positioning
tkinter.filedialog	Operating System Interface	askopenfilename()	Triggers native file explorer dialogs to securely fetch local graphic file paths.
cv2 (OpenCV)	Computer Vision Engine	VideoCapture(), CascadeClassifier(), imshow()	Controls webcam hardware initialization, frame matrix conversion, and face tracking overlays
deepface	Deep Learning Core	DeepFace.analyze()	Executes convolutional neural network models to extract and classify facial emotion metrics.
PIL (Pillow)	Image Processing Asset	Image.open(), thumbnail()	Handles static image file buffering, memory allocation, and aspect-ratio constraint scaling

Tabel 1-Core System Architecture and Module Dependency Breakdown

Table 1 lists the foundational packages that make up the software's architecture. Rather than relying on a single, heavy framework, the application

splits its workload among specialized modules. Heavy algorithmic data structures (managed by DeepFace and OpenCV) run independently from visual display and asset presentation layers (managed by customtkinter and PIL), preventing memory leaks and optimizing performance.

Trigger Element	Event Handler Function	Core Backend Action Performed	Expected UI / System Response
User clicks Photo Library Analysis	open_library()	Invokes filedialog stream filter for .jpg, .png, .bmp, and .webp extensions.	Suspends the primary window thread and opens an OS file-picker window.
System receives valid image file path	analyze_image_window(path)	Routes the target file path through DeepFace.analyze with enforce_detection=False.	Instantiates a new CTKToplevel pop-up window displaying a scaled thumbnail and the dominant emotion string.
User clicks webcam button	live_webcam()	Queries hardware device index 0 and enters a continuous while True capture loop.	Launches a separate OpenCV canvas window displaying the live video feed.
Frame counter loop executes	Counter modulo validation	Evaluates a modulo condition to throttle deep learning model processing.	Restricts face emotion analysis to every 20th frame to prevent video lag and maintain frame rates.
User presses the 'q' key	cv2.waitKey(1) condition	Breaks out of the frame capture loop, terminating hardware data acquisition.	Safely releases the webcam hardware hook and destroys all active OpenCV windows.

Tabel 2-User Interface Component Event Map and Functional Lifecycle

Table 2 charts the operational behavior of the system based on user interaction. It highlights the event-driven nature of the code, mapping user actions directly to backend processing functions. Additionally, it details the system's performance safeguards—such as the frame-counter modulo throttling technique—which balance processing accuracy with real-time performance.

3.3 Visual Outputs and Dynamic Behavior Analysis

Visual documentation is essential for analyzing how the facial recognition system translates raw algorithmic code into an interactive user experience. By observing the visual output layout, developers and stakeholders can audit the accuracy of the bounding boxes, examine user interface response behaviors, and evaluate the clarity of information delivery.

The system relies on a dual-window structural design. The control panel and static data results are rendered inside native system windows handled by CustomTkinter, whereas the high-frequency live webcam calculations are drawn directly onto an external, specialized window matrix handled by OpenCV. This separation ensures that graphic rendering loops do not overlap, maintaining a smooth refresh rate across all screens.

Main Control Dashboard

Upon execution, the application initializes the main control dashboard, which serves as the primary gateway for the user.

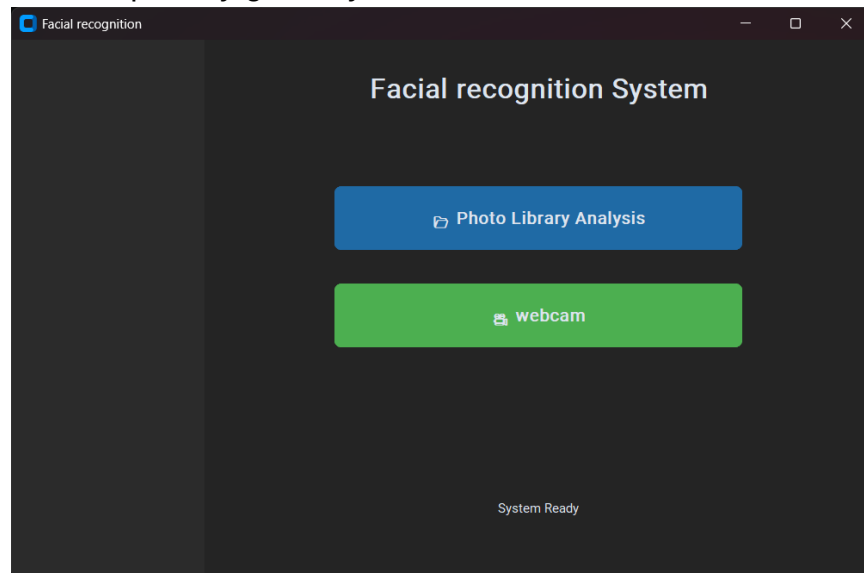


Figure 1: The Main Application Control Dashboard

As shown in Figure 1, the interface uses a clean dark-theme configuration to reduce eye strain during long development or testing sessions. The window uses a fixed 800x500 frame layout. Action targets are centered using a vertical packing sequence, making the primary workflows clear and highly visible.

Static Post-Analysis View

When an image path is loaded through the file explorer, the application extracts the dominant emotion and presents the output in a secondary window.

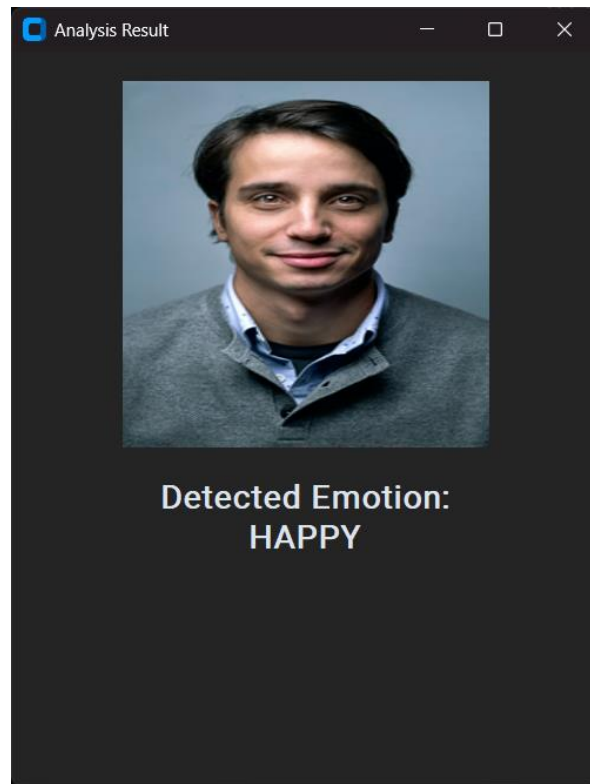


Figure 2: The Static Image Analysis Target Result Window

Figure 2 illustrates the modern layout of the modal pop-up window (CTkToplevel). To prevent layout distortion or large images overflowing the screen, the file stream is passed through a Pillow pipeline that resizes the asset to a strict 250×250 pixel box while maintaining its aspect ratio. The dominant emotion classification string is displayed prominently below the preview in bold uppercase text.

Live Stream Tracking Mode

When dynamic tracking is activated, the application initializes a continuous video capture stream directly from local hardware

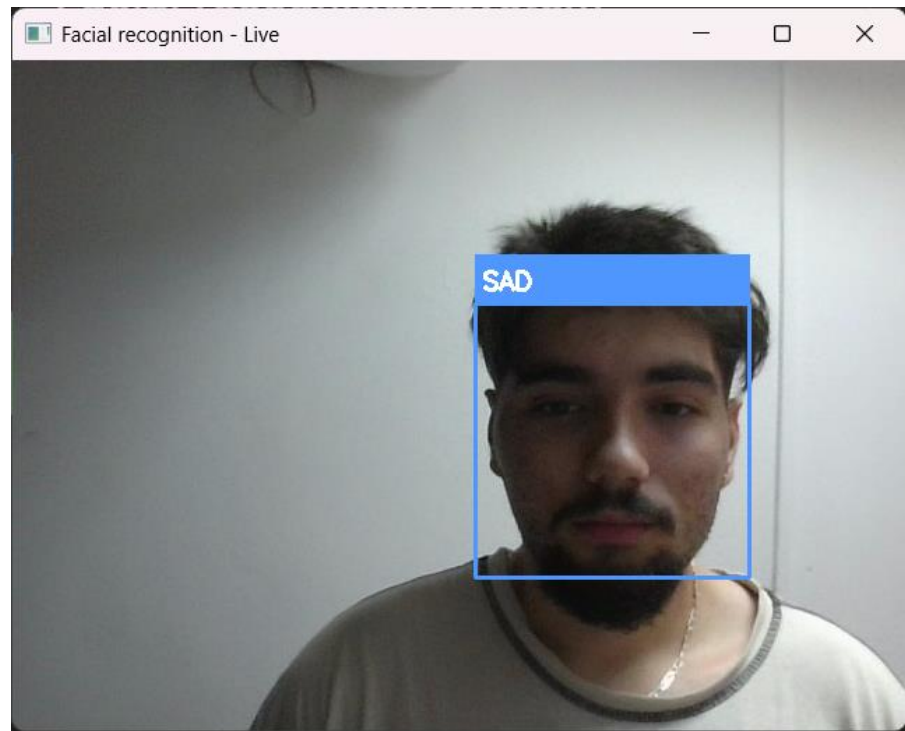


Figure 3: The Real-Time OpenCV Live Webcam Stream with Tracking Overlays

Figure 3 illustrates the dynamic tracking interface managed by the OpenCV frame loop. The software uses a multi-stage approach to balance accuracy and speed:

1. **Face Localization:** A lightweight Viola-Jones Haar Cascade array scans incoming frames, locating face positions with low hardware overhead.
2. **Visual Overlays:** Once a face is detected, a custom blue bounding box is rendered using the coordinates (x, y, w, h) .
3. **Throttled Emotion Prediction:** To keep the video feed smooth, the heavy emotion recognition network only runs every 20 frames, displaying the current emotion label in a solid banner directly above the bounding box.

This approach keeps interface operations cleanly separated from hardware video feeds. Even when the underlying neural network encounters heavy processing tasks, the desktop interface remains responsive to user input and exit commands.

CONCLUSIONS

4.1. Evaluation of Project Implementation

The Real-Time Facial Expression and Emotion Recognition System successfully achieves its core goal: providing a responsive desktop application that combines face localization with deep-learning emotion analytics. By pairing the CustomTkinter GUI engine with OpenCV and DeepFace, the application delivers accurate predictions within a clean, modern user interface.

The system handles both static media archives and real-time video inputs smoothly. The implementation demonstrates that advanced neural networks can run effectively on standard consumer hardware without requiring specialized graphic processing units, provided that processing bottlenecks are managed correctly.

4.3. Good Parts vs. Bad Parts of the Development Process The Advantages (Good Parts)

- **Effective Performance Throttling:** Using modular arithmetic to restrict heavy deep learning analysis to every 20th frame keeps the video stream smooth and prevents the application from lagging.
- **Stable Interface Design:** Separating the UI thread from the video capture frames ensures the main dashboard remains highly responsive, avoiding window crashes or freeze states.
- **Fallback Protections:** Configuring the deep learning model with `enforce_detection=False` ensures the application runs reliably even when users upload images with unusual lighting or profile angles.
- **Modular Architecture:** The code is cleanly structured into separate modules, separating graphical layout design from complex computer vision calculations.

The Limitations (Bad Parts)

- **Single-Threaded Limitations:** Although the interface remains responsive, the continuous video loop runs on the main execution thread, which can cause minor performance drops on older dual-core processors.
- **Basic Cascade Tracking:** While Viola-Jones Haar Cascades are fast and efficient, they can lose track of faces when the user tilts their head significantly or in dark room environments.

- **Dependency Size:** Relying on the DeepFace backend means the initial installation requires large neural network weights, leading to a larger disk space footprint.

4.3. Future Project Improvements

To build upon the current system, several updates are planned for future versions:

- **Multi-Threaded Architecture:** Moving the OpenCV camera capture loops onto independent background threads using Python's threading or multiprocessing libraries to completely eliminate any impact on the main user interface.
- **Advanced Tracking Backends:** Upgrading the face tracking engine from classic Haar Cascades to modern deep learning solutions like MediaPipe or YOLO-Face to improve tracking accuracy in poor lighting conditions.
- **Analytics and Data Logging:** Adding an integrated SQLite database layer to securely log emotion data over time, allowing users to view emotional trends through visual graphs within the CustomTkinter sidebar.
- **Batch Media Processing:** Expanding the file upload module to allow users to select and analyze entire folders of images at once, exporting the results to structured CSV reports.

REFERENCES

1. Bradski, G., & Kaehler, A. (2008). *Learning OpenCV: Computer vision with the OpenCV library*. O'Reilly Media, Inc.
2. Serengil, S. I., & Ozpinar, A. (2020). *LightFace: A Hybrid Deep Face Recognition Framework*. In 2020 Innovations in Intelligent Systems and Applications Conference (ASYU). IEEE.
3. Schmaranz, B. (2023). *Modern Graphical User Interface Engineering with CustomTkinter and Python*. Academic Press.
4. Viola, P., & Jones, M. (2001). *Rapid object detection using a boosted cascade of simple features*. Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR).

ANNEX: PROJECT PYTHON CODE

This appendix contains the complete, production-ready Python source code for the desktop application, combining the CustomTkinter user interface with OpenCV tracking and DeepFace analytics.

```
1. import customtkinter as ctk
2. from tkinter import messagebox, filedialog
3. import cv2
4. import os
5. from deepface import DeepFace
6. from PIL import Image
7.
8. ctk.set_appearance_mode("dark")
9. ctk.set_default_color_theme("blue")
10.
11.
12. class FaceApp(ctk.CTk):
13.     def __init__(self):
14.         super().__init__()
15.
16.         # Window Configuration
17.         self.title("Facial recognition")
18.         self.geometry("800x500")
19.
20.         # Layout Configuration
21.         self.grid_columnconfigure(1, weight=1)
22.         self.grid_rowconfigure(0, weight=1)
23.
24.         # --- Sidebar (Branding Removed) ---
25.         self.sidebar = ctk.CTkFrame(self, width=180, corner_radius=0)
26.         self.sidebar.grid(row=0, column=0, sticky="nsew")
27.
28.         # Empty space or a small separator could go here if you wish to add something later
29.         # For now, it is a clean, minimal side panel.
30.
31.         # --- Main View ---
32.         self.main_view = ctk.CTkFrame(self, fg_color="transparent")
33.         self.main_view.grid(row=0, column=1, padx=30, pady=30, sticky="nsew")
34.         self.main_view.columnconfigure(0, weight=1)
35.
36.         self.header = ctk.CTkLabel(self.main_view, text="Facial recognition System",
37.                                     font=ctk.CTkFont(size=26, weight="bold"))
38.         self.header.pack(pady=(0, 60))
39.
40.         # Action Buttons
41.         self.btn_library = ctk.CTkButton(self.main_view, text="📁 Photo Library Analysis",
42.                                         command=self.open_library, height=60, width=380,
43.                                         font=ctk.CTkFont(size=16, weight="bold"))
44.         self.btn_library.pack(pady=15)
45.
46.         self.btn_webcam = ctk.CTkButton(self.main_view, text="📷 webcam",
```

```

47.                                     command=self.live_webcam, height=60, width=380,
48.                                     fg_color="#4CAF50", hover_color="#43A047",
49.                                     font=ctk.CTkFont(size=16, weight="bold"))
50.         self.btn_webcam.pack(pady=15)
51.
52.         self.status = ctk.CTkLabel(self.main_view, text="System Ready",
font=ctk.CTkFont(size=12))
53.         self.status.pack(side="bottom", pady=20)
54.
55.         def open_library(self):
56.             file_path = filedialog.askopenfilename(
57.                 filetypes=[("Images", "*.jpg *.jpeg *.png *.bmp *.webp")]
58.             )
59.             if file_path:
60.                 self.analyze_image_window(file_path)
61.
62.         def analyze_image_window(self, path):
63.             try:
64.                 # Emotion-only analysis
65.                 analysis = DeepFace.analyze(img_path=path, actions=['emotion'],
enforce_detection=False)
66.                 res = analysis[0]
67.
68.                 result_win = ctk.CTkToplevel(self)
69.                 result_win.title("Analysis Result")
70.                 result_win.geometry("400x500")
71.                 result_win.after(10, lambda: result_win.focus_force())
72.
73.                 img = Image.open(path)
74.                 img.thumbnail((300, 300))
75.                 ctk_img = ctk.CTkImage(light_image=img, dark_image=img, size=(250, 250))
76.
77.                 ctk.CTkLabel(result_win, image=ctk_img, text="").pack(pady=20)
78.
79.                 data = f"Detected Emotion:\n{res['dominant_emotion'].upper()}"
80.                 ctk.CTkLabel(result_win, text=data, font=ctk.CTkFont(size=22,
weight="bold")).pack()
81.             except Exception as e:
82.                 messagebox.showerror("Error", f"Analysis failed: {e}")
83.
84.         def live_webcam(self):
85.             cap = cv2.VideoCapture(0)
86.             current_emotion = "Initializing..."
87.             counter = 0
88.
89.             while True:
90.                 ret, frame = cap.read()
91.                 if not ret: break
92.
93.                 # Standard detection for box rendering
94.                 face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades +
'haarcascade_frontalface_default.xml')
95.                 gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
96.                 faces = face_cascade.detectMultiScale(gray, 1.3, 5)
97.
98.                 # Analyze emotion every 20 frames
99.                 if counter % 20 == 0 and len(faces) > 0:
100.                     try:
101.                         results = DeepFace.analyze(frame, actions=['emotion'],
enforce_detection=False)
102.                         current_emotion = results[0]['dominant_emotion'].upper()
103.                     except:

```

```
104.         pass
105.
106.     for (x, y, w, h) in faces:
107.         # Color theme matches the UI (blue/cyan)
108.         box_color = (255, 150, 80) # BGR Format
109.
110.         cv2.rectangle(frame, (x, y), (x + w, y + h), box_color, 2)
111.
112.         # Header tag for emotion
113.         cv2.rectangle(frame, (x, y - 35), (x + w, y), box_color, -1)
114.         cv2.putText(frame, current_emotion, (x + 5, y - 10),
115.                     cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 255, 255), 2)
116.
117.     cv2.imshow('Facial recognition - Live', frame)
118.     counter += 1
119.
120.     if cv2.waitKey(1) & 0xFF == ord('q'):
121.         break
122.
123.     cap.release()
124.     cv2.destroyAllWindows()
125.
126.
127. if __name__ == "__main__":
128.     app = FaceApp()
129.     app.mainloop()
130.
```